

Interactive Constraint Explanations for Human–Agent Graph Colouring:

A Comparative Study of Perspective and Representation

Abstract

Effective collaboration between humans and AI systems in constraint-satisfaction tasks depends not only on finding valid solutions, but on helping human collaborators understand the constraint structure that governs feasibility. We present a platform for studying how different representations of constraint information affect human understanding, workload, and task performance in a graph-colouring task with partial observability. Rather than requiring the human to negotiate with agents via dialogue, the redesigned interface exposes the implications of the human’s choices in real time: agents act as *constraint analysers* that continuously compute and display the feasible solution space induced by the human’s current node-colour assignments. Two design dimensions define four experimental conditions: *perspective* (user-centric vs. agent-centric framing of constraint information) and *representation* (formulaic logical expressions vs. natural-language summaries generated by an LLM). We describe the formal model, the real-time computation engine, the four interface conditions in detail, and a user-study design comparing objective convergence metrics against subjective workload, clarity, and trust.

1 Introduction

Humans working alongside AI agents in constraint-satisfaction tasks face a fundamental interpretive challenge: understanding how their own decisions constrain the options available to their collaborators. In classical approaches, agents negotiate solutions with human partners through structured or conversational dialogue. While such systems have been studied extensively, dialogue introduces its own cognitive demands — interpreting agent moves, formulating responses, tracking negotiation state — which are orthogonal to the underlying optimisation problem.

This paper re-centres the research question from *how should agents negotiate with humans?* to *how should agents explain constraint structure to humans?* We hypothesise that the principal difficulty in human–agent coordination is not reaching agreement through dialogue but helping humans understand the constraint space in which they are operating.

We investigate this question using a graph-colouring task in which the human directly manipulates node colours and the system continuously reveals the implications of those choices. Agents no longer act as negotiating partners; instead, they act as constraint analysers. As the human adjusts their cluster’s colours, the system recomputes — in real time — which configurations remain feasible for each agent cluster, and displays this information through one of four interface conditions.

The two independent variables are:

1. **Perspective:** whether constraint information is framed relative to the *human’s choices* (user-centric) or relative to the *agent’s remaining options* (agent-centric).
2. **Representation:** whether constraints are expressed as *formulaic logical statements* or as *natural-language summaries* produced by an LLM.

The contributions of this paper are:

1. A formal model of interactive constraint visualisation for distributed graph colouring, including definitions of feasible sets, domain projections, and infeasibility conditions.
2. A real-time constraint computation engine that exactly evaluates agent feasibility given arbitrary partial human assignments.

3. Detailed specifications of four interface conditions spanning two design dimensions.
4. A user-study design for comparing these conditions on objective and subjective measures.

2 Related Work

2.1 Distributed Constraint Optimisation

Distributed Constraint Optimisation Problems [?] provide a canonical framework for multi-agent coordination under coupled local constraints. Classic algorithms such as DPOP [?] and Max-Sum [2] achieve globally optimal or bounded-suboptimal solutions through structured message passing. Our work adopts the DCOP formulation as the experimental task but shifts the focus from algorithm design to human understanding of the feasibility structure that such algorithms exploit.

2.2 Mixed-Initiative and Human–AI Collaboration

Mixed-initiative systems allow control to be shared adaptively between humans and agents [?]. Within optimisation, this raises the question of how much constraint information the human needs to make useful contributions. Approaches range from fully automated solving with post-hoc explanation to interactive solving where humans manipulate partial solutions and agents fill in the rest [?]. Our interface occupies the latter end: the human has full control over their cluster and agents respond by exposing the downstream implications rather than issuing demands.

2.3 Explanations in AI Systems

The explainability literature distinguishes *global* explanations (properties of the model as a whole) from *local* explanations (reasons for a specific output) [?]. Closest to our setting is contrastive explanation [?], which answers the question “why this outcome rather than another?” User-centric constraint framing is a form of contrastive explanation: “because you chose $h_4 = \text{blue}$, the agent is forced into a specific subset of configurations.” We operationalise and compare such contrastive framing against a more direct display of remaining options (agent-centric).

2.4 Constraint Visualisation

Several interactive constraint-solving tools have been developed to support human understanding of feasibility [?]. These systems typically expose constraint propagation results visually, allowing users to observe which variable domains contract as they fix values. Our system extends this approach to a distributed multi-agent setting where one agent’s feasible set depends on another participant’s choices, and where the explanation interface is itself the experimental variable.

2.5 LLMs for Explanation and Summarisation

LLMs have demonstrated strong capability in summarising complex technical content in accessible natural language [?]. Recent work has applied LLMs to explain planning and optimisation outputs to lay audiences [1]. A key design question is whether LLM-generated summaries preserve sufficient precision for constraint reasoning, or whether formulaic expressions are required for accurate interpretation. Our experiment directly compares these two representation styles under controlled conditions.

3 Problem Formulation

3.1 The Graph-Colouring Task

We use the same distributed graph-colouring task as the original study. The global graph $G = (V, E)$ is partitioned among three participants — two artificial agents a_1, a_2 and one human h — each controlling a disjoint cluster of

nodes: $V = V_{a_1} \sqcup V_h \sqcup V_{a_2}$. Each participant assigns a colour from a shared domain $\mathcal{C}$ (default: red, green, blue) to each of their nodes. The global conflict penalty is:

$$J(x) = \sum_{(u,v) \in E} \mathbf{1}[x_u = x_v] \quad (1)$$

where $x : V \rightarrow \mathcal{C}$ is the global assignment.

In the default experiment: $|V| = 15$ nodes (five per cluster), $|\mathcal{C}| = 3$, approximately 4–6 inter-cluster boundary edges, and a cluster topology of $V_{a_1} \leftrightarrow V_h \leftrightarrow V_{a_2}$ (the two agent clusters are coupled only through the human’s cluster).

3.2 Partial Observability

Each participant has *local* visibility: they fully observe their own cluster and can see the boundary-neighbour nodes that share an edge with their cluster, but have no knowledge of nodes further inside other clusters. Formally, participant p ’s visible external nodes are:

$$\partial V_p = \{v \notin V_p \mid \exists u \in V_p : (u, v) \in E\}$$

No topology beyond ∂V_p is ever exposed to participant p , whether in the UI or in any generated explanation.

3.3 Partial Human Assignment

In the new interface the human’s configuration is modelled as a *partial* assignment, since nodes may be left unspecified (displayed as grey) while the human explores possibilities:

$$\alpha_h : V_h \rightarrow \mathcal{C} \cup \{\perp\} \quad (2)$$

where $\perp$ represents an unassigned (grey) node. The *effective domain* of α_h is $\text{dom}(\alpha_h) = \{v \in V_h \mid \alpha_h(v) \neq \perp\}$.

This differs from the original negotiation study, where the human always maintained a complete assignment. Allowing grey nodes is essential for exploration: the human can fix a subset of their choices and observe feasibility before committing to the rest.

3.4 Agent Feasibility Sets

Given the human’s partial assignment α_h , the set of *feasible configurations* for agent cluster a is:

$$S_a(\alpha_h) = \{x_a \in \mathcal{C}^{|V_a|} \mid J_a(x_a, \alpha_h) = 0\} \quad (3)$$

where $J_a(x_a, \alpha_h)$ counts conflicts on edges incident to V_a , treating unassigned boundary-neighbour nodes as imposing no conflict constraint. The system computes $S_a(\alpha_h)$ *exactly* by exhaustive enumeration over $|\mathcal{C}|^{|V_a|}$ candidates (tractable for $k = 3, n = 5$: 243 combinations).

The cluster is *feasible* if $S_a(\alpha_h) \neq \emptyset$ and *infeasible* if $S_a(\alpha_h) = \emptyset$. Infeasibility means no valid global colouring exists given the human’s current choices; the interface notifies the human and proposes a minimal repair (Section 4.4).

3.5 Constraint Projections

Because the full feasible set $S_a(\alpha_h)$ is a set of complete cluster assignments, it is difficult to display directly. The interface instead presents two *projections*.

Domain projection (agent-centric). For each agent node $u \in V_a$, the set of colours that u can still adopt:

$$D_u(\alpha_h) = \{c \in \mathcal{C} \mid \exists x_a \in S_a(\alpha_h) : x_a(u) = c\} \quad (4)$$

This is the marginal projection of $S_a(\alpha_h)$ onto node u .

Consequence projection (user-centric). For each human node $v \in V_h$ whose colour is fixed, the subset of agent configurations that are reachable:

$$S_a(\alpha_h \mid v = c) = \{x_a \in S_a(\alpha_h) \mid \text{consistent with } \alpha_h(v) = c\} \quad (5)$$

The interface presents this as a logical description of which agent-node assignments are jointly implied (or excluded) by the human's choice of $\alpha_h(v) = c$.

4 System Architecture

4.1 Graph Interface

The participant sees a graph canvas displaying:

- All nodes in the human cluster V_h , coloured according to α_h (grey if unassigned).
- The boundary-neighbour nodes ∂V_h from each agent cluster, displayed but not directly manipulable by the human.
- Edges connecting nodes, with conflicting edges highlighted.

The human cycles through colours (red, green, blue, grey) by clicking any node in V_h . Available colours are $\mathcal{C} = \{\text{red, green, blue}\}$; grey represents the unassigned state $\perp$.

4.2 Real-Time Constraint Engine

Whenever the human changes a node colour, the system executes the following pipeline:

1. Update α_h with the new assignment.
2. For each agent $a \in \{a_1, a_2\}$, recompute $S_a(\alpha_h)$ by exhaustive enumeration.
3. For each agent node u , recompute $D_u(\alpha_h)$ from $S_a(\alpha_h)$.
4. Compute user-centric consequence descriptions for each fixed human node.
5. Update all constraint panels with the new information.

This pipeline completes within milliseconds for the default graph size, so updates appear instantaneous. There is no turn structure and no messaging interface.

4.3 Constraint Panels

The interface contains two types of constraint display panels, each populated according to the active experimental condition:

Node-level panels. Each node (human or agent) may display an annotation showing its constraint status. In agent-centric conditions, agent nodes display their domain $D_u(\alpha_h)$. In user-centric conditions, human nodes display the consequence sets induced by their current colour choice.

Global feasibility panel. A summary panel at the top of the screen shows the feasibility status of each agent cluster. Possible states are: FEASIBLE ($S_a \neq \emptyset$), INFEASIBLE ($S_a = \emptyset$), and UNCONSTRAINED (no boundary edges fixed yet, so all configurations remain possible).

4.4 Infeasibility Repair Proposals

When $S_a(\alpha_h) = \emptyset$, the system identifies a minimal set of human nodes whose reassignment would restore feasibility. Formally, it finds the smallest set $R \subseteq \text{dom}(\alpha_h)$ such that:

$$S_a(\alpha_h \mid_{V_h \setminus R}) \neq \emptyset$$

This is computed by iterating over subsets of fixed human nodes in order of increasing size. The interface then proposes specific colour changes for the nodes in R , framed according to the active condition (as a logical statement or as a natural-language suggestion).

4.5 LLM Summary Generation

In the LLM-representation conditions (Conditions 3 and 4), the formulaic constraint expressions are passed to a language model to produce natural-language summaries. The LLM receives:

- The precomputed domain set $D_u(\alpha_h)$ or consequence description for the relevant node.
- The current human assignment α_h .
- A system prompt instructing it to summarise the constraint information in accessible natural language without inventing additional constraints.

The LLM *does not perform any reasoning or constraint computation*. It acts as a *presentation layer only*: the constraint information it receives is exact and precomputed; its task is solely to render it more readable. All LLM calls are logged with the input constraint data and output text, enabling post-hoc verification that no hallucinated constraints were introduced.

5 Experimental Conditions

The study uses a 2×2 factorial design with two within-subjects factors:

Perspective *User-centric* (constraints framed relative to the human’s choices) vs. *Agent-centric* (constraints framed relative to agent nodes’ remaining options).

Representation *Formulaic* (logical expressions) vs. *Natural language* (LLM-generated summaries of the same information).

This produces four conditions. All conditions share the same real-time computation engine and the same graph interface; only the constraint panels differ.

5.1 Condition 1: User-Centric Formulaic

Human nodes display logical expressions describing the set of agent configurations that are jointly induced by the human’s current colour choices.

Display format. Each fixed human node v with $\alpha_h(v) = c$ displays the constraint:

Because: $v = c$
Requires: (Agent configs consistent with α_h)

When $\alpha_h(v) = c$ forces a specific joint agent assignment, this is displayed as a conjunction. When multiple compatible agent configurations remain, they are listed as a disjunction:

Because: $h4 = \text{Blue}$
Means: ($a5 = \text{Red AND } a4 = \text{Green}$) OR ($a5 = \text{Green AND } a4 = \text{Red}$)

Design intent. This condition provides the most informationally precise display, directly linking the human’s decisions to agent outcomes. It is expected to be cognitively demanding, particularly when many agent configurations remain compatible.

5.2 Condition 2: Agent-Centric Formulaic

Agent nodes display the set of colours they may still adopt, expressed as a logical formula.

Display format. Each agent node u displays its domain projection $D_u(\alpha_h)$:

a5: {Red, Green}

When a node’s domain is a singleton ($|D_u| = 1$), the colour is forced and the display uses the wording “must be”. Conditional entries — colours available only under specific human assignments — are flagged separately:

Possible: a5 = Green OR Red
Conditional: a5 = Blue IF h4 $\neq$ Blue

Design intent. This condition highlights remaining degrees of freedom for agent nodes, making it straightforward to see how much flexibility the system retains. It does not, however, directly express the causal link from human decisions to agent constraints.

5.3 Condition 3: User-Centric Natural Language

The same user-centric consequence information as Condition 1 is presented, but rendered into natural language by the LLM rather than as a logical formula.

Display format. Human nodes display a plain-English summary:

Because h4 is Blue, the neighbouring agent nodes are restricted
to two configurations: either a5 is Red and a4 is Green,
or a5 is Green and a4 is Red.

When few constraints are active, the summary may convey that the agent remains largely unconstrained:

Your current choice leaves most agent configurations open.
The neighbouring nodes are only weakly constrained.

Design intent. This condition tests whether LLM-rendered descriptions reduce the cognitive load of interpreting formulaic logical expressions while preserving the user-centric causal framing.

5.4 Condition 4: Agent-Centric Natural Language

The same agent-centric domain information as Condition 2 is rendered into natural language by the LLM.

Display format. Agent nodes display plain-English descriptions of their feasible colours:

a5 can currently be Red or Green. Blue would only become
available if h4 changed to a different colour.

Design intent. This condition tests whether natural-language framing of remaining options is sufficient for users to guide their colour choices efficiently, without the additional cognitive overhead of interpreting consequence-directed formulae.

6 Experimental Design

6.1 Task

Participants must colour all nodes in their cluster such that a valid global colouring exists — that is, such that $S_{a_1}(\alpha_h) \neq \emptyset$ and $S_{a_2}(\alpha_h) \neq \emptyset$ simultaneously. Participants may:

- Change any node colour freely at any time.
- Leave nodes grey (unassigned) while exploring possibilities.
- Consult the constraint panels at any time.

The task ends when the participant confirms that a feasible assignment exists and submits their final colouring. There is no dialogue, no agent messaging, and no turn structure.

6.2 Conditions and Design

We use a within-subjects design: each participant completes the task under all four conditions, counterbalanced across participants using a 4×4 Latin square to control for learning and order effects. A different graph instance is used for each condition to prevent transfer of specific solution knowledge.

6.3 Graph Instances

Four distinct graph instances are generated, matched for:

- $|V| = 15$, five nodes per cluster.
- $|\mathcal{C}| = 3$ colours.
- 4–6 inter-cluster boundary edges.
- Chromatic number equal to 3, ensuring a valid 3-colouring exists but requiring coordination between clusters.
- Comparable solution density: number of valid colourings within 10% across instances.

Graph instances are generated by random sampling with these constraints and validated by exhaustive enumeration.

6.4 Procedure

The session proceeds as follows:

1. **Introduction** (10 min): task explanation, graph colouring rules, interface tutorial using a practice graph not used in the main conditions.
2. **Condition blocks** (4×15 min): one block per condition, separated by 2-minute breaks. Each block includes:
 - a. Brief instruction screen showing the current condition’s interface layout.
 - b. Task on the assigned graph instance.
 - c. Post-block questionnaire (workload, clarity, control).
3. **End-of-session battery** (15 min): full NASA-TLX, trust questionnaire, preference ranking across all four conditions.

Total session duration is approximately 90 minutes.

6.5 Participants

Approximately 32 participants ($n \geq 28$ after exclusions), aged 18+, with no prior experience with graph colouring problems. Participants are recruited from a university participant pool. Inclusion criteria: normal or corrected-to-normal colour vision (essential for graph display), native or near-native English (for natural-language conditions), and ability to complete a 90-minute session.

6.6 Measures

Objective.

- **Time to feasibility:** wall-clock time from task start to confirmed feasible submission.
- **Node adjustments:** total number of colour changes made during the task.
- **Infeasible visits:** number of timesteps during which the global configuration was infeasible ($S_{a_1} = \emptyset$ or $S_{a_2} = \emptyset$).
- **Exploration breadth:** number of distinct colour assignments visited across human nodes, as a proxy for search strategy.

Subjective (post-block).

- **Workload:** NASA-TLX [?] six subscales, administered after each block.
- **Perceived clarity:** custom 5-item scale measuring how clearly the interface communicated the constraints.
- **Perceived control:** adapted from [?]; degree to which the participant felt in control of the outcome.

Subjective (end of session).

- **Trust in system guidance:** post-task trust questionnaire adapted from [?], assessing trust in the constraint information displayed.
- **Mode preference:** paired-comparison ranking across all four conditions, eliciting which interface the participant would prefer for sustained use.
- **Open comments:** optional free-text field for qualitative feedback.

7 Expected Results and Hypotheses

The 2×2 design permits main effects and interactions between perspective and representation. We state the primary hypotheses below.

H1 (*Representation – Workload*) LLM-generated natural-language summaries (Conditions 3 and 4) will yield lower NASA-TLX scores than formulaic conditions (Conditions 1 and 2), because they reduce the syntactic parsing overhead of logical expressions.

H2 (*Perspective – Convergence*) Agent-centric conditions (Conditions 2 and 4) will yield faster time to feasibility, because directly displaying remaining degrees of freedom gives the human more immediately actionable information about which node changes are most effective.

H3 (*Perspective – Understanding*) User-centric conditions (Conditions 1 and 3) will yield higher perceived clarity scores, because linking the human’s choices directly to agent outcomes makes the causal structure of the constraint system more legible.

H4 (*Interaction*) The benefit of natural-language rendering will be larger for user-centric conditions, because the consequence descriptions produced in user-centric mode are structurally more complex (disjunctions over joint agent assignments) and thus benefit more from summarisation.

Table 1: Expected qualitative performance of the four conditions. VH = Very High, H = High, M = Medium, L = Low, VL = Very Low.

Attribute	C1 (UC-F)	C2 (AC-F)	C3 (UC-NL)	C4 (AC-NL)
Time to feasibility (low = fast)	H	L	M	VL
Node adjustments	H	M	M	L
Infeasible visits	M	L	M	VL
Cognitive workload	VH	H	M	L
Perceived clarity	H	M	VH	M
Perceived control	M	H	M	H
Trust in guidance	H	H	M	M
Mode preference	L	M	M	VH

H5 (Trust) Trust in system guidance will be higher for formulaic conditions, as the precise logical statements are more verifiable than LLM-generated prose, which participants may perceive as potentially imprecise.

Table 1 summarises expected qualitative performance across the four conditions.
(UC = User-Centric; AC = Agent-Centric; F = Formulaic; NL = Natural Language.)

We expect Condition 4 (agent-centric, natural-language) to offer the best overall trade-off between speed and usability, and Condition 1 (user-centric, formulaic) to provide the highest precision at the cost of high cognitive load. The primary scientific question is whether user-centric or agent-centric framing more strongly improves human understanding when natural language is available.

8 Conclusion

This study shifts the focus of human–agent coordination research from dialogue-based negotiation to constraint explanation. Rather than asking how agents and humans should exchange proposals, we ask how AI systems should present the constraint structure of a problem so that humans can make effective decisions. The redesigned platform isolates two dimensions of this question — perspective and representation — and provides an experimental infrastructure for testing their effects on understanding, workload, and performance.

The interface designed here — interactive constraint visualisation with real-time feasibility computation — represents a novel class of human–AI interaction that could generalise beyond graph colouring to any cooperative constraint-satisfaction task where a human makes partial assignments and agents must adapt.

Future work includes extending the approach to larger DCOP instances where exhaustive enumeration must be replaced by approximate projection, studying how participants develop constraint-reasoning strategies over repeated trials, and investigating richer explanation modalities such as counterfactual comparisons (“if you had chosen green instead of blue...”).

References

- [1] Weihang Guo, Zachary Kingston, and Lydia E Kavraki. Castl: Constraints as specifications through llm translation for long-horizon task and motion planning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11957–11964. IEEE, 2025.
- [2] Alex Rogers, Alessandro Farinelli, Ruben Stranders, and Nicholas R Jennings. Bounded approximate decentralised coordination via the max-sum algorithm. *Artificial Intelligence*, 175(2):730–759, 2011.

Technical Appendix

Formal Definitions, Algorithms, and Implementation Details

This appendix provides complete formal definitions, pseudocode for all algorithms, implementation details, and example interface traces. It is intended as a self-contained technical reference for the system described in the main paper.

Contents

1	Introduction	1
2	Related Work	2
2.1	Distributed Constraint Optimisation	2
2.2	Mixed-Initiative and Human–AI Collaboration	2
2.3	Explanations in AI Systems	2
2.4	Constraint Visualisation	2
2.5	LLMs for Explanation and Summarisation	2
3	Problem Formulation	2
3.1	The Graph-Colouring Task	2
3.2	Partial Observability	3
3.3	Partial Human Assignment	3
3.4	Agent Feasibility Sets	3
3.5	Constraint Projections	3
4	System Architecture	4
4.1	Graph Interface	4
4.2	Real-Time Constraint Engine	4
4.3	Constraint Panels	4
4.4	Infeasibility Repair Proposals	5
4.5	LLM Summary Generation	5
5	Experimental Conditions	5
5.1	Condition 1: User-Centric Formulaic	5
5.2	Condition 2: Agent-Centric Formulaic	6
5.3	Condition 3: User-Centric Natural Language	6
5.4	Condition 4: Agent-Centric Natural Language	6
6	Experimental Design	7
6.1	Task	7
6.2	Conditions and Design	7
6.3	Graph Instances	7
6.4	Procedure	7
6.5	Participants	8
6.6	Measures	8
7	Expected Results and Hypotheses	8
8	Conclusion	9
A	Formal Problem Definition	12
A.1	Graph Colouring DCOP	12
A.2	Notation Summary	12

B	Real-Time Constraint Computation	13
B.1	Feasibility Set Enumeration	13
B.2	Domain Projection	13
B.3	Conditional Domain Entries	13
B.4	Real-Time Update Pipeline	13
C	Infeasibility Repair	13
D	LLM Summary Generation	15
D.1	Input Construction	15
D.2	LLM Call Specification	15
D.3	Hallucination Logging	16
E	Interface Condition Specifications	16
E.1	Condition 1: User-Centric Formulaic	16
E.2	Condition 2: Agent-Centric Formulaic	16
E.3	Condition 3: User-Centric Natural Language	16
E.4	Condition 4: Agent-Centric Natural Language	16
F	Example Interaction Traces	17
F.1	Scenario	17
G	Graph Instance Generation	17

A Formal Problem Definition

A.1 Graph Colouring DCOP

Definition A.1 (Graph Colouring DCOP). A graph colouring DCOP is a tuple $\langle G, \mathcal{C}, \mathcal{P}, \mathcal{V} \rangle$:

- $G = (V, E)$: undirected graph with node set V , edge set E .
- $\mathcal{C}$: finite colour domain, $|\mathcal{C}| = k$ (default $k = 3$).
- $\mathcal{P} = \{a_1, a_2, h\}$: participants (agents a_1, a_2 , human h).
- $\mathcal{V} : \mathcal{P} \rightarrow 2^V$: cluster partition, $\bigsqcup_{p \in \mathcal{P}} \mathcal{V}(p) = V$.

Definition A.2 (Boundary and Internal Nodes). For each participant p with cluster V_p :

$$\begin{aligned} \partial V_p &= \{v \in V \setminus V_p \mid \exists u \in V_p : (u, v) \in E\} && \text{(visible external nodes)} \\ B_p &= \{u \in V_p \mid \exists v \in \partial V_p : (u, v) \in E\} && \text{(boundary nodes of } p) \\ I_p &= V_p \setminus B_p && \text{(internal nodes of } p) \end{aligned}$$

Definition A.3 (Partial Human Assignment). The human's configuration is a function $\alpha_h : V_h \rightarrow \mathcal{C} \cup \{\perp\}$, where $\perp$ denotes an unassigned (grey) node. The effective domain is $\text{dom}(\alpha_h) = \{v \in V_h \mid \alpha_h(v) \neq \perp\}$. The *colour projection* of α_h onto a set $U \subseteq V_h$ is $\alpha_h|_U = \{v \mapsto \alpha_h(v) \mid v \in U \cap \text{dom}(\alpha_h)\}$.

Definition A.4 (Agent Feasibility Set). For agent a with cluster V_a and boundary $\partial V_a \cap V_h \subseteq V_h$, the feasibility set under partial human assignment α_h is:

$$S_a(\alpha_h) = \{x_a \in \mathcal{C}^{|V_a|} \mid J_a(x_a, \alpha_h|_{\partial V_a}) = 0\}$$

where $J_a(x_a, \alpha_h|_{\partial V_a})$ counts conflicts on edges incident to V_a , treating unassigned boundary neighbours as imposing no constraint.

Definition A.5 (Domain Projection). For agent node $u \in V_a$, the *domain projection* under α_h is:

$$D_u(\alpha_h) = \{c \in \mathcal{C} \mid \exists x_a \in S_a(\alpha_h) : x_a(u) = c\}$$

Definition A.6 (Consequence Set). For human node $v \in \text{dom}(\alpha_h)$ with $\alpha_h(v) = c$, the *consequence set* for agent a is:

$$\text{Cons}_{a,v}(\alpha_h) = \{x_a \in S_a(\alpha_h) \mid \text{consistent with } \alpha_h(v) = c\}$$

In practice this is simply $S_a(\alpha_h)$ restricted to configurations that do not conflict with the fixed colour of v 's boundary neighbours in V_a .

A.2 Notation Summary

- α_h : partial human assignment; $\alpha_h(v) = \perp$ if v is grey.
- $\text{dom}(\alpha_h)$: fixed (non-grey) human nodes.
- $S_a(\alpha_h)$: feasibility set for agent cluster a .
- $D_u(\alpha_h)$: feasible colour domain for agent node u .
- $J(x) = \sum_{(u,v) \in E} \mathbf{1}[x_u = x_v]$: global conflict penalty.
- $J_a(x_a, \alpha_h|_{\partial V_a})$: local penalty for agent a 's edges.
- $k = |\mathcal{C}|$: number of colours (default 3).
- $n = |V_a|$: nodes per cluster (default 5).
- $\epsilon = 10^{-6}$: numerical tolerance for zero-penalty checks.

B Real-Time Constraint Computation

B.1 Feasibility Set Enumeration

Algorithm 1 ComputeFeasibilitySet: exact enumeration of $S_a(\alpha_h)$

Require: Agent a , partial assignment α_h , graph (V, E) , colour domain $\mathcal{C}$

```

1:  $\text{bnd} \leftarrow \partial V_a \cap \text{dom}(\alpha_h)$  {boundary nodes with fixed human colours}
2:  $S \leftarrow \emptyset$ 
3: for all  $(c_1, \dots, c_n) \in \mathcal{C}^{|V_a|}$  { $3^5 = 243$  candidates} do
4:    $x_a \leftarrow \{v_i \mapsto c_i\}_{i=1}^n$ 
5:    $\text{conflicts} \leftarrow 0$ 
6:   for all  $(u, v) \in E$  with  $u \in V_a$  do
7:     if  $v \in \text{bnd}$  and  $x_a[u] = \alpha_h(v)$  then
8:        $\text{conflicts} \leftarrow \text{conflicts} + 1$ 
9:     end if
10:    if  $v \in V_a$  and  $x_a[u] = x_a[v]$  then
11:       $\text{conflicts} \leftarrow \text{conflicts} + 1$ 
12:    end if
13:  end for
14:  if  $\text{conflicts} = 0$  then
15:     $S \leftarrow S \cup \{x_a\}$ 
16:  end if
17: end for
18: return  $S$ 

```

B.2 Domain Projection

Algorithm 2 ComputeDomainProjection: $D_u(\alpha_h)$ for all agent nodes

Require: Agent a , feasibility set $S_a(\alpha_h)$

```

1: Initialise  $D_u \leftarrow \emptyset$  for all  $u \in V_a$ 
2: for all  $x_a \in S_a(\alpha_h)$  do
3:   for all  $u \in V_a$  do
4:      $D_u \leftarrow D_u \cup \{x_a[u]\}$ 
5:   end for
6: end for
7: return  $\{u \mapsto D_u \mid u \in V_a\}$ 

```

B.3 Conditional Domain Entries

A colour $c \in \mathcal{C} \setminus D_u(\alpha_h)$ is *conditionally available* for node u if there exists an alternative partial human assignment α'_h that differs from α_h in at most one node and makes $c \in D_u(\alpha'_h)$. This is used in Condition 2 to populate the conditional entries of the domain display.

B.4 Real-Time Update Pipeline

C Infeasibility Repair

Algorithm 3 ComputeConditionalDomains: single-change conditional entries

Require: Agent a , partial assignment α_h , domain $D_u(\alpha_h)$

```
1: Cond  $\leftarrow \{u \mapsto \{\} \mid u \in V_a\}$ 
2: for all  $v \in \text{dom}(\alpha_h)$  with  $v \in \partial V_a$  do
3:   for all  $c' \in \mathcal{C} \setminus \{\alpha_h(v)\}$  do
4:      $\alpha'_h \leftarrow \alpha_h[v \mapsto c']$ 
5:      $S' \leftarrow \text{ComputeFeasibilitySet}(a, \alpha'_h)$ 
6:      $D' \leftarrow \text{ComputeDomainProjection}(a, S')$ 
7:     for all  $u \in V_a$  do
8:       for all  $d \in D'[u] \setminus D_u(\alpha_h)$  do
9:          $\text{Cond}[u] \leftarrow \text{Cond}[u] \cup \{(d, v \neq \alpha_h(v))\}$ 
10:      end for
11:    end for
12:  end for
13: end for
14: return Cond
```

Algorithm 4 OnNodeColourChange: pipeline triggered by human interaction

Require: Updated node v , new colour c' (or $\perp$ if reset to grey), current α_h

```
1:  $\alpha_h[v] \leftarrow c'$ 
2: for all  $a \in \{a_1, a_2\}$  (in parallel) do
3:    $S_a \leftarrow \text{ComputeFeasibilitySet}(a, \alpha_h)$ 
4:    $D_a \leftarrow \text{ComputeDomainProjection}(a, S_a)$ 
5:   if active condition is agent-centric then
6:      $\text{Cond}_a \leftarrow \text{ComputeConditionalDomains}(a, \alpha_h, D_a)$ 
7:   else
8:      $\text{Cons}_a \leftarrow \text{ComputeConsequenceSets}(a, \alpha_h, S_a)$ 
9:   end if
10:   $\text{Feasible}_a \leftarrow (S_a \neq \emptyset)$ 
11: end for
12: Update feasibility panel with  $\text{Feasible}_{a_1}, \text{Feasible}_{a_2}$ 
13: Update node annotations according to active condition
14: if active condition uses LLM representation then
15:   Queue LLM summary calls for changed annotations (Section D)
16: end if
```

Algorithm 5 InfeasibilityRepair: minimal node set to restore feasibility

Require: Agent a , partial assignment α_h

```
1: fixed  $\leftarrow \text{dom}(\alpha_h) \cap \partial V_a$  {human nodes adjacent to agent  $a$ }
2: for  $r = 1, 2, \dots, |\text{fixed}|$  do
3:   for all  $R \subseteq \text{fixed}$  with  $|R| = r$  do
4:      $\alpha'_h \leftarrow \alpha_h|_{V_h \setminus R}$  {release  $R$  to unassigned}
5:     if  $\text{ComputeFeasibilitySet}(a, \alpha'_h) \neq \emptyset$  then
6:       return  $R$  with suggested colours for each node in  $R$ 
7:     end if
8:   end for
9: end for
10: return "No valid colouring exists for this graph." {only if graph is not 3-colourable; guaranteed not to happen by construction}
```

The suggested colours for each node $v \in R$ are drawn from a feasible configuration of the smallest $S_a(\alpha'_h)$ found, projected onto the boundary neighbours of v .

D LLM Summary Generation

D.1 Input Construction

For each node annotation that changes in an LLM-representation condition, the system constructs a structured prompt as follows.

Agent-centric (Condition 4). The prompt provides:

```

1 {
2   "node": "a5",
3   "feasible_colours": ["Red", "Green"],
4   "conditional_colours": [
5     {"colour": "Blue", "condition": "h4 != Blue"}
6   ],
7   "human_assignment": {"h4": "Blue", "h3": "Red"}
8 }
```

System instruction: “Describe, in one or two plain-English sentences, what colours node a5 can take and under what conditions. Do not introduce constraints not present in the data.”

User-centric (Condition 3). The prompt provides the consequence set for the changed human node:

```

1 {
2   "human_node": "h4",
3   "human_colour": "Blue",
4   "agent": "a1",
5   "feasible_agent_configs": [
6     {"a4": "Green", "a5": "Red"},
7     {"a4": "Red", "a5": "Green"}
8   ]
9 }
```

System instruction: “Summarise, in one or two plain-English sentences, what agent configurations are implied by this human node choice. Do not introduce constraints not present in the data.”

D.2 LLM Call Specification

All LLM calls use the chat completions API with:

- temperature = 0.2 (low variance for reproducibility).
- max_tokens = 80 (enforces brevity).
- Model: configurable; default is a capable instruction-following model.

The LLM response is used as-is. If the API call fails, the system falls back to the formulaic display for that node, and logs a warning. This is the only fallback permitted, and is justified because the fallback is a more informative display, not a degraded one.

D.3 Hallucination Logging

All LLM inputs and outputs are logged to `results/llm_explanations.jsonl`, including:

- The exact structured input (feasibility set or domain projection).
- The LLM prompt and response.
- A post-hoc verification flag set by comparing the LLM output against the structured input for any mentioned constraint that is absent from the input.

Post-hoc analysis of this log enables detection of hallucinated constraints that were presented to participants during the study.

E Interface Condition Specifications

E.1 Condition 1: User-Centric Formulaic

Each human node $v \in \text{dom}(\alpha_h)$ with $\alpha_h(v) = c$ displays:

If $|\text{Cons}_{a,v}(\alpha_h)| = 1$ (forced assignment):

```
1 Because: h4 = Blue
2 Forces: a5 = Red AND a4 = Green
```

If $|\text{Cons}_{a,v}(\alpha_h)| > 1$ (disjunctive):

```
1 Because: h4 = Blue
2 Allows: (a5 = Red AND a4 = Green)
3         OR (a5 = Green AND a4 = Red)
4 [+ 3 more configurations]
```

When more than 4 configurations exist, the display shows the first 4 and a count of remaining ones.

If $|\text{Cons}_{a,v}(\alpha_h)| = 0$ (infeasible):

```
1 Because: h4 = Blue
2 INFEASIBLE: No valid agent configuration exists.
3 [Change suggestion displayed below]
```

E.2 Condition 2: Agent-Centric Formulaic

Each agent node $u \in V_a$ displays:

```
1 a5
2 Possible:      Green, Red
3 Conditional:   Blue IF h4 != Blue
```

When $|D_u(\alpha_h)| = 0$, the node annotation displays [INFEASIBLE] in red.

E.3 Condition 3: User-Centric Natural Language

Human node annotations are replaced by LLM-generated summaries of the consequence sets (Section D). Layout and positioning are identical to Condition 1.

E.4 Condition 4: Agent-Centric Natural Language

Agent node annotations are replaced by LLM-generated summaries of the domain projections (Section D). Layout and positioning are identical to Condition 2.

F Example Interaction Traces

The following traces illustrate the system’s behaviour across all four conditions for an identical sequence of human interactions on a sample graph.

F.1 Scenario

Nodes h_3, h_4, h_5 are in the human cluster; a_4, a_5 are boundary nodes of agent a_1 . Boundary edge (h_4, a_5) exists. The human begins with all nodes grey.

Step 1: Human sets $h_4 = \text{Blue}$. Condition 1 output (user-centric formulaic):

```
1 h4 [Blue]
2 Because: h4 = Blue
3 Allows: (a5 = Red AND a4 = Green)
4         OR (a5 = Green AND a4 = Red)
```

Condition 2 output (agent-centric formulaic):

```
1 a5: Possible: {Red, Green}    Conditional: Blue IF h4 != Blue
2 a4: Possible: {Red, Green, Blue}
```

Condition 3 output (user-centric natural language):

```
1 h4 [Blue]
2 Setting h4 to Blue means the neighbouring agent must choose
3 between two configurations: a5 Red with a4 Green, or a5 Green
4 with a4 Red.
```

Condition 4 output (agent-centric natural language):

```
1 a5: Can currently be Red or Green. Blue would only be possible
2   if h4 were changed to a different colour.
3 a4: Can be any colour.
```

Step 2: Human sets $h_4 = \text{Red}$ (changes colour). Agent feasibility set updates; all conditions update immediately. In Condition 2, a_5 ’s domain now excludes Red and adds Blue as a directly possible colour.

Step 3: Human sets $h_3 = \text{Blue}$, creating an infeasible state. Global feasibility panel updates to show INFEASIBLE for a_1 . In all conditions, the repair proposal is displayed:

```
1 INFEASIBLE: No valid colouring for Agent 1.
2 Suggested fix: change h3 to Green or Red.
```

G Graph Instance Generation

The four graph instances used in the study are generated by the following procedure:

The bounds $N_{\min}$ and $N_{\max}$ are set to ensure instances are neither trivially easy (many valid colourings) nor excessively constrained (very few). Exact values are determined during piloting.

Algorithm 6 GenerateMatchedGraphInstances

Require: Number of instances $M = 4$, nodes per cluster $n = 5$, colours $k = 3$

```
1: instances  $\leftarrow \emptyset$ 
2: while |instances| <  $M$  do
3:   Sample random intra-cluster edges for each cluster (Erdős–Rényi,  $p = 0.4$ )
4:   Sample 4–6 inter-cluster boundary edges between  $B_h$  and  $\partial V_h$ 
5:   Verify  $G$  is  $k$ -colourable by exhaustive search
6:   Count the number of valid  $k$ -colourings  $N_{\text{valid}}$ 
7:   if  $N_{\text{valid}} \geq N_{\text{min}}$  and  $N_{\text{valid}} \leq N_{\text{max}}$  then
8:     instances  $\leftarrow$  instances  $\cup \{G\}$ 
9:   end if
10: end while
11: Verify that valid-colouring counts across all instances are within 10% of each other
12: return instances
```
